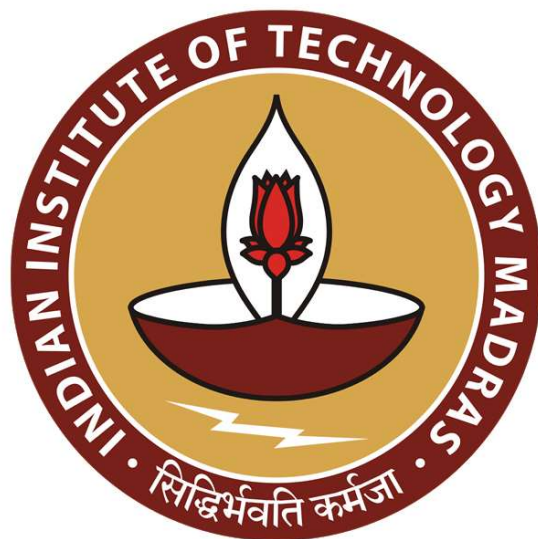
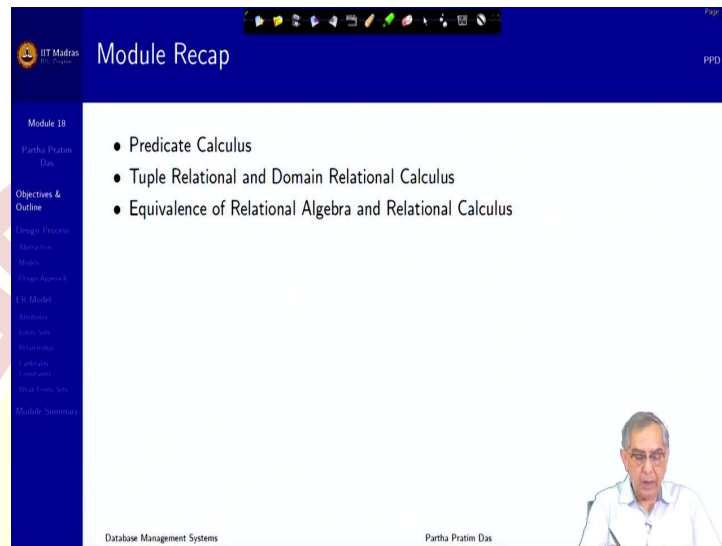




IIT Madras
ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
Entity- Relationship Model-1

(Refer Slide Time: 0:27)



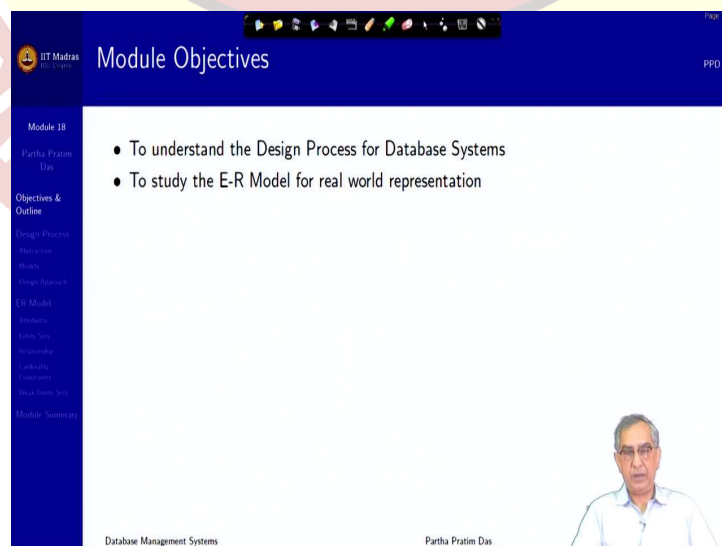
The screenshot shows a presentation slide titled "Module Recap" for "Module 18" in the "Database Management Systems" course. The slide lists three topics:

- Predicate Calculus
- Tuple Relational and Domain Relational Calculus
- Equivalence of Relational Algebra and Relational Calculus

A video feed of Professor Partha Pratim Das is visible in the bottom right corner of the slide.

Welcome to Module 18 of Database Management Systems course in the IIT Madras online B.Sc program. In the last two modules we have taken a look at the relational algebra as well as in the last module we looked at the calculus. And we will interchangeably keep using these notions as we go forward to design the database as well as, as we work on designing the query.

(Refer Slide Time: 0:56)



The screenshot shows a presentation slide titled "Module Objectives" for "Module 18" in the "Database Management Systems" course. The slide lists two objectives:

- To understand the Design Process for Database Systems
- To study the E-R Model for real world representation

A video feed of Professor Partha Pratim Das is visible in the bottom right corner of the slide.

From this module we will start discussing the design process. How do you design a database system for a particular application? And what are the tools that are available so we will talk about E-R model entity relationship model for real world representation.

(Refer Slide Time: 01:18)

Module Outline

Module 1B
Partha Pratim Das

Objectives & Outline

- Design Process
- E-R Model
 - Entity and Entity Set
 - Relationship
 - Cardinality
 - Attributes
 - Weak Entity Sets

Database Management Systems

Partha Pratim Das

So, these are the contents.

(Refer Slide Time: 1:22)

Design Process

Database Management Systems

Partha Pratim Das

IT Madras
Page 2

What is Design?

Module 18
Partha Pratim Das
Objectives & Outline
Design Process
Introduction
Models
Design Process
Data Model
Database
Query Language
Database Design
Database Implementation
Database Security
Database Recovery
Database Administration
Module Summary

A Design:

- Satisfies a given (perhaps informal) functional specification
- Conforms to limitations of the target medium
- Meets implicit or explicit requirements on performance and resource usage
- Satisfies implicit or explicit design criteria on the form of the artifact
- Satisfies restrictions on the design process itself, such as its length or cost, or the tools available for doing the design

Database Management Systems Partha Pratim Das

Now, talking about the design process let us, in general try to see that what is design? You know what we mean by designing, naturally when you design you design for a requirement. You require you have a requirement specification or functional specification or something that you want to design for and meet those requirements. This could be a simple academic application like instructors, courses, students, scores attendant, grades and all that so you will have to know the total functional specification for it.

You need to know who are the stakeholders? What kind of constraints or what kind of rules business rules will exist in that, or it could be a financial application, it could be a banking situation, it could be any of these, but the first thing of a design is to know the functional specification that is the key area.

And then we will have to do a design which conforms to the limitations of the target medium. Because depending on the requirement if you have the requirement of an academic program and you say that I need a supercomputer to run this obviously that will not be possible. But for a small database application for a small store, it may be fine to have a single user PC to run that program.

So, you have to know what the specifications are and what the target system is an accordingly you have to make that design. And you have to implicitly or explicitly meet all requirements of performance resource use usage, some will have lot of concurrent use like their banking application response time of under a second, some will have a single user application some will have a very distributed say nation wise distributed application like your COWIN application or Aarogya Setu application and so on.

All of them have big databases behind them. So, you have to satisfy implicitly or explicitly the design criteria that you set for yourself. And in the form of artifacts, you must satisfy them and satisfy the restrictions that are possible. So, these are these are very global gross thoughts that you will have in your mind when you go about doing the design.

(Refer Slide Time: 4:10)

Role of Abstraction

- *Disorganized Complexity* results from
 - *Storage (STM) limitations* of human brain – an individual can simultaneously comprehend of the order of seven, plus or minus two chunks of information
 - *Speed limitations* of human brain – it takes the mind about five seconds to accept a new chunk of information
- **Abstraction** provides the major tool to handle Disorganized Complexity by *chunking information*
- Ignore inessential details, deal only with the generalized, idealized model of the world

Consider: A binary number 110010101001 ✓

Hard to remember. Right?

Try the octal form: (110)(010)(101)(001) ⇒ 6251 ✓

Or the hex form: (1100)(1010)(1001) ⇒ CA9 ✓

Now, at the beginning of the design process let me just remind you that design process has two key components to start with one is, the abstraction and the other is the model. Abstraction is you can look at the same information at multiple levels of granularity or representation. And we need the abstraction so that we can mentally handle it.

Like human brain has a limited capacity it is a the short term memory as it is called the it has storage limitation at a time we can remember about 5 to 7 items together not more than that, with as a speed limitation of the human brain. So, we always need to abstract things down so that we have the information but it needs lot less memory time and effort to understand that information.

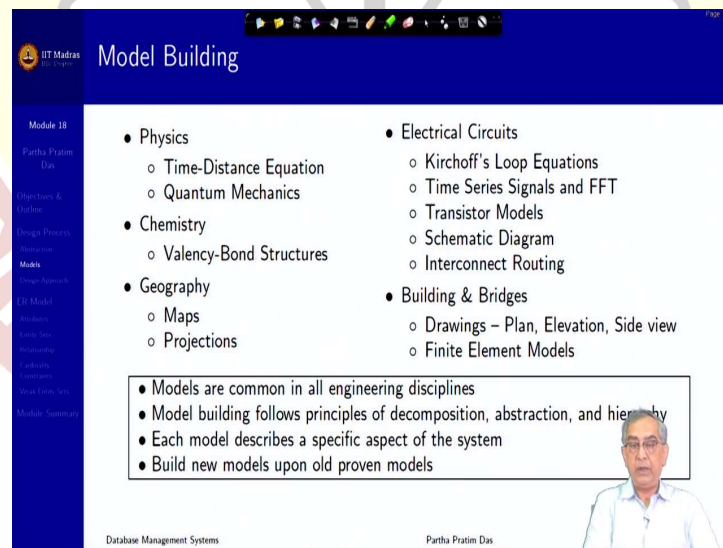
And of course, in terms of doing that you ignore a lot of details which may not be relevant for it. So, this is again a mental process which has to stay with you that always focus on what is a required information and abstract on them and work with them while you are you will ignore all the remaining details. For example, in the in the student academic application when you are dealing with courses, grades, and so on the say the salary of the instructor or say the date of birth of the student are unnecessary information.

Because you do not need them in this context. So, your abstraction is just your around courses and academy connection. Whereas when you are working with the departmental budget you are really not that concerned about which courses the instructor is teaching. But you are very concerned about what is the salary of the instructor what is the infrastructure cost to support that instructor and so on.

So, that is kind of the angle of the abstraction in particular. So, just to give you an example of abstraction here I have to show you a simple binary string and if I asked you to remember this string by just looking at it naturally most of us would not be able to do it. But if we group these bits in terms of triplet and encoded in octal form it is 6251. So, you will just need to look at it once and you will know.

So, this and this both have the same information but the abstraction is different by 6251 you can remember it very easily. And you can reconstruct the detailed information also in this case you do not need to ignore anything. Similarly, if you group these as 4 bits it is CA9 in hexadecimal form. So, these are like you can just look at it and will remember so that is the value of abstraction. And you will always have to keep that in mind that what is the appropriate level at which we abstract.

(Refer Slide Time: 7:18)



Model Building

Module 18
Partha Pratim Das

- Physics
 - Time-Distance Equation
 - Quantum Mechanics
- Chemistry
 - Valency-Bond Structures
- Geography
 - Maps
 - Projections
- Electrical Circuits
 - Kirchoff's Loop Equations
 - Time Series Signals and FFT
 - Transistor Models
 - Schematic Diagram
 - Interconnect Routing
- Building & Bridges
 - Drawings – Plan, Elevation, Side view
 - Finite Element Models

• Models are common in all engineering disciplines
• Model building follows principles of decomposition, abstraction, and hierarchy
• Each model describes a specific aspect of the system
• Build new models upon old proven models

Database Management Systems Partha Pratim Das

The second aspect is there is a significant importance of building models. We are always dealing with the real world but we have to deal with the real world in terms of certain mathematics in terms of certain computing algorithms in terms of certain computer system architecture and so on.

So, we need to make models of the real world which mimic the real world. So, that if I want something to happen in the real world, I do a corresponding thing in the model and the it will happen in the real world. If I do something in the model it is similar things should corresponding things should happen in the real world. So, just to give you an examples this is not specific to database design.

If you think about physics, you have studied Newton's laws of motion that is a model of motion. You have studied that $S=UT+\frac{1}{2}AT^2$ that is a S is a distance travel U is the initial velocity, D is the time elapsed. And A is acceleration then this is the distance travelled. Now, that distance travelled is a physical entity. How much the train has gone from in this time from this station.

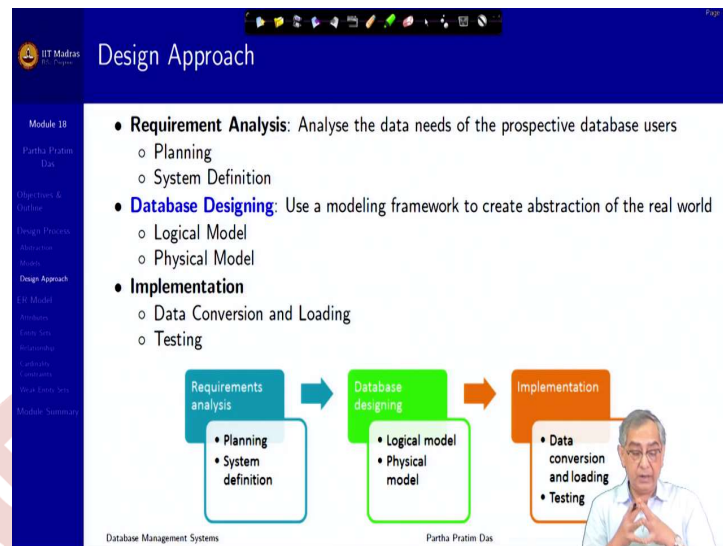
So, should it have reached the next station but you can reason about it talk about it analyse it by using this formula. So, that is the model of time distance. So, if you look into various kinds of things, you have already done in your school and high school you have quantum mechanics, I mean in physics is full of models you have quantum mechanics to describe certain real world the situation which we cannot see by non-common eye.

Chemistry is full of models balance will be structure for elements and compounds. Geography is models of maps projections electrical circuits many of you would have electrical background maybe. Kirchhoff's laws transistor models schematic diagram. So, again if you see that there are several models of for the same thing. And several models look at the abstraction from several angles.

So, when you are doing Kirchhoff's loop equation you are interested in voltage and current. Whereas when you are doing the schematic diagram, you are interested just to understand how the connectivities are what is going to happen when you are doing interconnected routing you are trying to see how it looks on the PCB. The same electrical component electrical circuit is being viewed in several different ways and these are the different models.

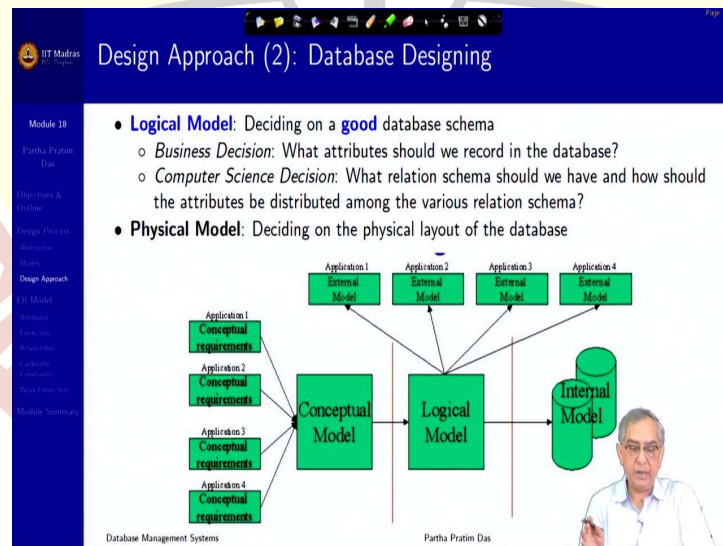
And building model is the most important thing so before we start with the design process, we need to learn an appropriate framework for modelling the real world that is of consequence to our design requirements.

(Refer Slide Time: 10:13)



So, the overall design process of a database I can divide into three parts one is requirement analysis how do you plan and that is you acquire the requirements you put the specifications define the system. Then there is database design in the middle which has logical and physical model design. And finally, you go into the implementation data conversion and test and so on.

(Refer Slide Time: 10:38)



So, we will right now focus on the logical model part of it that is database designing has a logical part, which we talked about is a conceptual part and the other is a physical part of bits and bytes and pages and links and so on. So, we will come to the we will start with the design process with database designing with a with the logical model. We will assume that the

requirements have been captured. We will talk about requirements capture as some time later. We will talk about the physical model some time later.

(Refer Slide Time: 11:14)

Design Approach (3): Database Designing: Logical Model

- **Entity Relationship Model**
 - Models an enterprise as a collection of entities and relationships
 - ▷ *Entity*: A distinguishable "thing" or "object" in the enterprise
 - Described by a set of attributes
 - ▷ *Relationship*: An association among multiple entities
 - Represented by an *Entity-Relationship* or *ER Diagram*
- **Database Normalization (Chapter 8)**
 - Formalize what designs are bad, and test for them

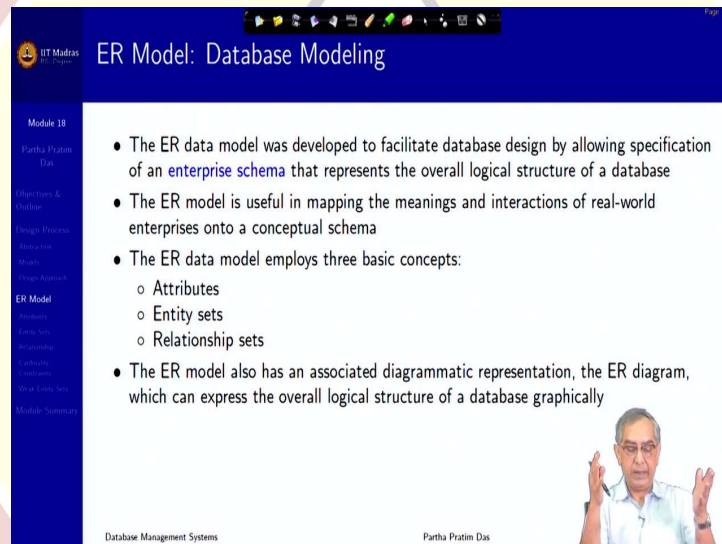
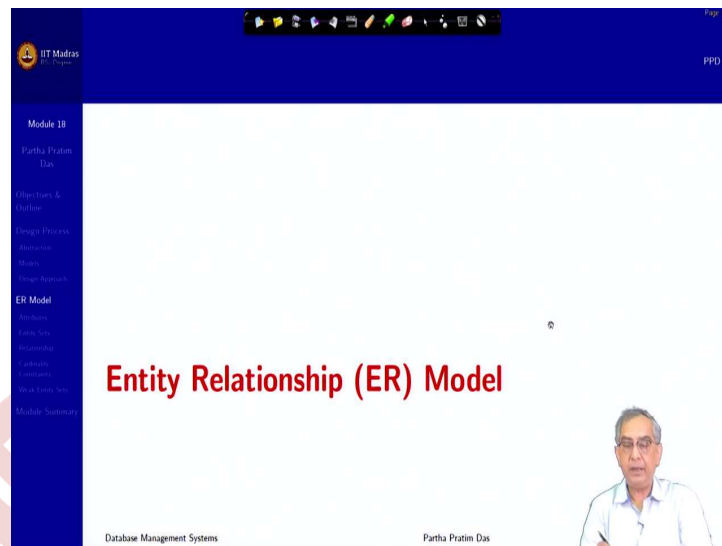
Database Management Systems Partho Pratim Das

And focusing on the logical model we will go with a specific framework to design the capture the logical model which is called the entity relationship model. We talked about this loosely earlier to an entity is basically the anything that you can distinguish as every separate object or thing. And this distinction is done in terms of a set of attributes which might be enough to distinguish the entity. So, we have seen how we can distinguish instructors.

So, the collection of instructors is an entity set and every instructor is an entity who can be distinguished through the properties or the attributes that it has. And then we have relationships between multiple entity sets. So, there are entity set of instructor, their entity set of courses and there is an relationship between them which says it teaches. So, entity relationship model which often is actually represented in terms of a diagram is typically a framework to capture this kind of information.

And once that is done then we will go into actually getting into relational tables and then we will go into database normalization process which will come as separate modules in a different week. So, we are going to get into the entity relationship model.

(Refer Slide Time: 12:44)

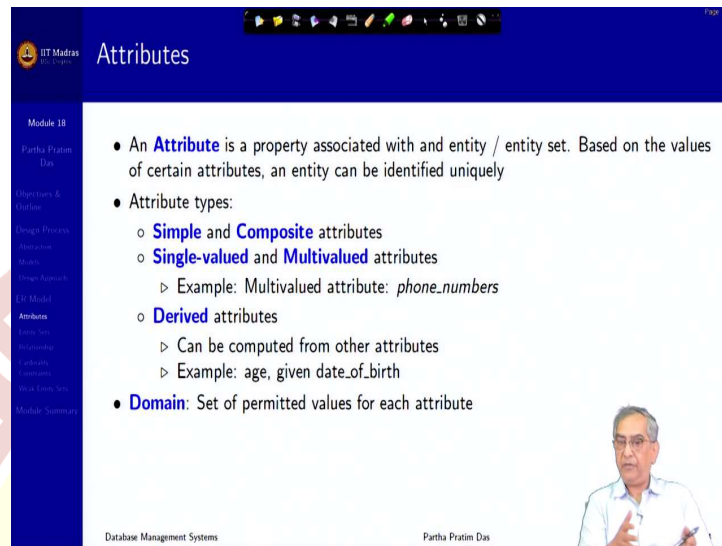


So, the idea is very simple that I want to model the requirements of the of an enterprise of which I have got the specification. And first what we are trying to do is just captured the conceptual requirements not getting into, what are the queries that we are going to support, what is the timing that should be required, what whether it is concurrent or non-concurrent and so on so forth.

We are not looking into all those we are just trying to capture the basic concept of the enterprise to the at most detail as possible. And for that we need three things we need attributes which describe different elements in the whole real world in the enterprise real world. These described by these attributes are entities and a set of similar entities having the same set of attributes will

form entity sets and then we have relationships which are also sets which are between two or more entity sets.

(Refer Slide Time: 13:52)



Attributes

- An **Attribute** is a property associated with an entity / entity set. Based on the values of certain attributes, an entity can be identified uniquely
- Attribute types:
 - **Simple** and **Composite** attributes
 - **Single-valued** and **Multivalued** attributes
 - ▷ Example: Multivalued attribute: *phone_numbers*
 - **Derived** attributes
 - ▷ Can be computed from other attributes
 - ▷ Example: age, given *date_of_birth*
- **Domain**: Set of permitted values for each attribute

Database Management Systems | Partha Pratim Das

So, this so just formally speaking what is an attribute is a property associated with entity set. And based on certain values of the attributes every entity must be identifiable uniquely. Now, an attribute may be simple or a composite attribute we will just explain a little bit more in terms of what is composite attributes simple attribute is a number, is a var char, is a character, is a string and so on so forth.

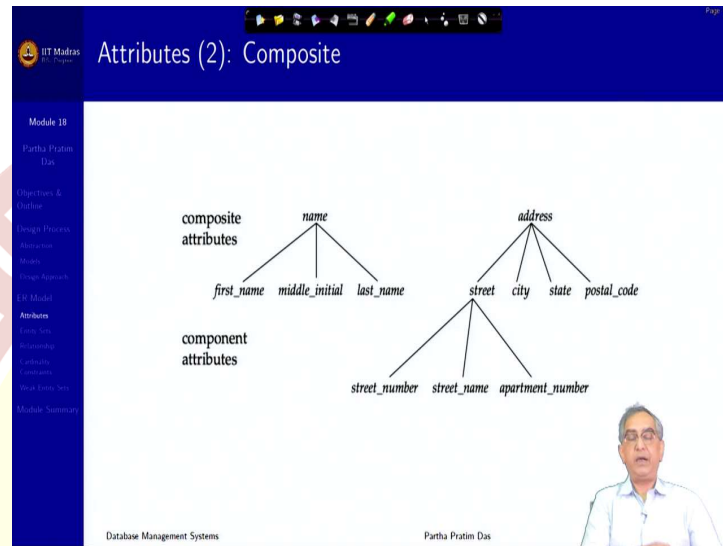
Then an attribute could be single valued or multi valued that is what it means is for a particular entity a course in attribute of that entity it is possible that it takes multiple values. For example, if we are modelling persons then which very likely that the person will have more than 1 mobile number. Nowadays people are keeping 2 3 mobile numbers some they use for normal calls, some they use for WhatsApp and so on.

So, in that case it I mean we have to see how we can key because there is only one field say for the phone number. But if the person has multiple phone numbers and if we want to maintain that then how do we handle that. So, this is the basically the requirement of a multivalued attribute. And there are certain attributes which are called derived which are which cannot be kept directly like you I want to keep the age of a person.

You cannot keep the age of the person because the day you keep it the next day it has become bad because the age has changed. So, what do you keep is the date of birth and from the date of birth on the day you want to know the age you can compute the age. So, you actually are

interested in the age but you are not keeping age you are keeping date of birth so you are deriving age from the date of birth. So, these are called derived attributes. So, these are the different types of attributes you will have and a domain is a set of permitted values for each attribute.

(Refer Slide Time: 15:50)



Now, coming to the composite attribute composite attribute is one which has more than one component, for example name typically in a person's name. There is a first name there is a last name and often there is a middle name. Now, this each one of this would be a string possibly and these three together forms the attribute notion of name because if a name differs in any one of these three.

Then it is a different name possibly or we may have rules that no in these conditions it is not. For example, can you say that the middle name is not of consequence in identification just the first name and last name will matter but the fact is that it is not one data entity simple entity which describes the attribute it has multiple. For example, in terms of address is typically composed of street city state postal code and so on.

Street in turn will be composite in terms of street number, street name, apartment number, and so on. So, even date could be considered as a composite attribute because it has a day, it has a month, it has a year. But if you are working in a see the system like sequel which has date as a built-in primitive type itself. Then we will consider date for that system for that modelling to be a simple attribute.

But if you are just working on a system which has say numbers to represent Numeric to represent then a date also is a composite attribute. So, we will have to see that in case of a composite attribute how do we actually model it and represent them.

(Refer Slide Time: 17:39)

Entity Sets

- An **entity** is an object that exists and is distinguishable from other objects.
 - Example: specific person, company, event, plant
- An **entity set** is a set of entities of the same type that share the same properties.
 - Example: set of all persons, companies, trees, holidays
- An entity is represented by a set of attributes; i.e., descriptive properties possessed by all members of an entity set.
 - Example:

instructor = (ID, name, street, city, salary)

course = (course_id, title, credits)
- A subset of the attributes form a **primary key** of the entity set; that is, uniquely identifying each member of the set.
 - Primary key of an entity set is represented by underlining it

Database Management Systems | Partha Pratim Das

Now, entity sets are clearly saying sets of distinguishable objects qualified by the attributes and you have already seen that instructor course all these we have been using for long. So, you represent the entity set by the list of attributes particular order is not important. And you with an underline you show what is a primary key. You all know what is the meaning of primary key primary key is that one or set of attributes which can uniquely identify every entity in that set.

(Refer Slide Time: 18:18)

Entity Sets - instructor and student

instructor_ID	instructor_name
76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

student-ID	student_name
98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

Database Management Systems | Partha Pratim Das

So, you underline that or the set of attributes. So, these are this is typically the entity sets this instructor ID and name student ID and name we have seen quite a bit of this already.

(Refer Slide Time: 18:28)

Relationship Sets

- A **relationship** is an association among several entities
Example:
44553 (Peltier) advisor 22222 (Einstein)
student entity relationship set instructor entity
- A **relationship set** is a mathematical relation among $n \geq 2$ entities, each taken from entity sets
$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

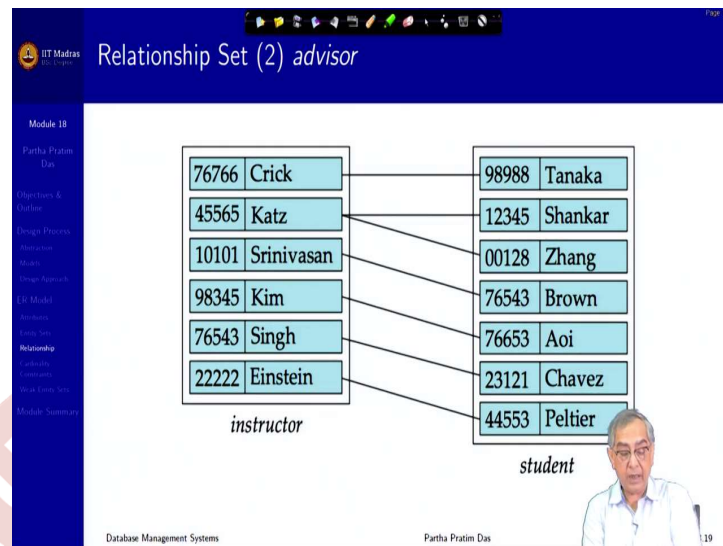
where $(e_1, e_2, \dots, e_n)$ is a relationship.
 - Example: $(44553, 22222) \in \text{advisor}$

Database Management Systems Partha Pratim Das 18

Relationships in contrast are what is defined between say initially let us assume between two entity sets. So, we say we have students, we have instructors and we are interested to define a relationship of advisor. Which says that the if the Peltier is related to Einstein, it means that Einstein is the advisor for a supervisor for Peltier. So, this will mean that advisor is kind of a relationship set which has a primary key of the student entity.

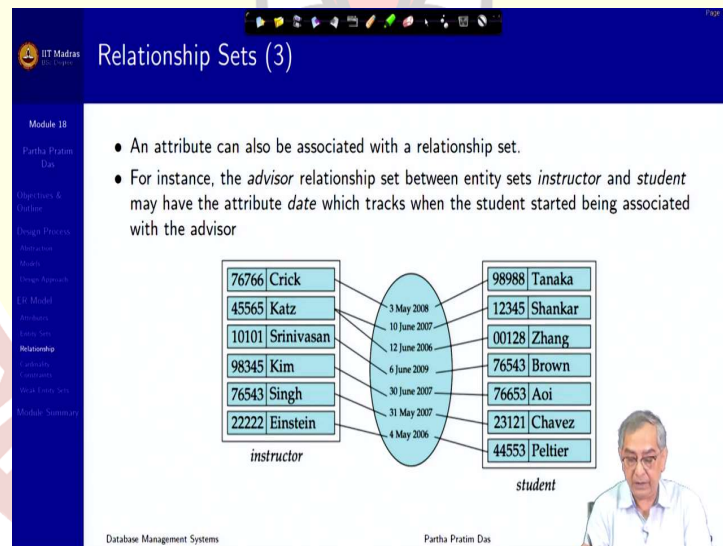
And the primary key of instructor entity denoting that these two are related. So, these are the relationships and you can in this is I am talking about two entity sets this could in general be for any n entity sets $e_1, e_2, \dots, e_n$ and in now you can see this is basically a domain relational calculus kind of representation. So, this n tuple is the relationship between this and different entity sets.

(Refer Slide Time: 19:39)



So, this is this link arrows show the relationship between instructor and student as advisors so Crick is advisor for Tanaka, Katz is a advisor for Shankar and Zhang and so on so forth.

(Refer Slide Time: 19:55)



Now, it is possible that in relationship it is not just the linkage but the relationship itself could have an attribute. For example, you want to in the advisor you want to maintain the information that well Crick is the advisor of Tanaka and I want to know the date from which Crick has become the advisor of Tanaka. So, you can put that as an attribute on that relation also. So, that is that is a new concept coming we talked about attributes only for entities. But attributes could be with an association with of the relationship.

(Refer Slide Time: 20:32)

Relationship Set (4): Degree

- Binary relationship
 - involves two entity sets (or degree two).
 - most relationship sets in a database system are binary.
- Relationships between more than two entity sets are rare. Most relationships are binary
 - Example: *students* work on research projects under the guidance of an *instructor*.
 - relationship *proj_guide* is a ternary relationship between *instructor*, *student*, and *project*

Database Management Systems Partha Pratim Das

We will see this then it depends on the degree whether the relationship is binary or it is more than binary, ternary, quaternary or so on. Most relationships sets in that the database are binary and that is what is preferred and we often if we have a situation of eternity in relationship or more we try to break it down into binary relationship maybe. For example, if we are talking about a project guide relationship it is just not just not a simple guide as we had earlier.

But it is a project guide. So, then you need to know three things you need to know the instructor that will be the guide, you need to know the student for whom the guide is, and you need to know the project in which the instructor is guiding the student. In a different project a different instructor may guide the student and so on so forth. So, this is a three-way relation ternary relation and we will have to be able to represent those. So, that is what is called the degree.

(Refer Slide Time: 21:40)

Attributes (3): Redundant

- Suppose we have entity sets:
 - *instructor*, with attributes: *ID*, *name*, *dept_name*, *salary*
 - *department*, with attributes: *dept_name*, *building*, *budget*
- We model the fact that each instructor has an associated department using a relationship set *inst_dept*
- The attribute *dept_name* appears in both entity sets. Since it is the primary key for the entity set *department*, it replicates information present in the relationship and is therefore redundant in the entity set *instructor* and needs to be removed
- BUT: When converting back to tables, in some cases the attribute gets reintroduced, as we will see later

Database Management Systems Partha Pratim Das

We will see in that in some cases and this will come out more in the as we unfold the design process that through the relationship many times certain attributes may become redundant. For example, instructor here has a department name and department of course has department name which is the key and the details. Now, to set a relationship if we say that there is a works for relationship between the instructor and the department.

Then it is through this relationship it is known what is the department of the instructor. So, it is it may be okay to drop this department name from the instructor table. You can because this that relationship we will talk of this ID which is a instructor this department name works for is a relationship which says what is the department of the instructor. So, it with I mean we can see that this actually this is this kind of attributes are often called redundant attribute.

Now, whether or not this is redundant depends on the other context of relationships and so on. For example, if department name was enough and there was no other details given though if there was no department entity set then obviously this is not redundant. And you will see that if we drop it because of redundancy oftentimes when we convert this E-R model back into the relational tables you might get this attribute back in the table for different reasons.

(Refer Slide Time: 23:25)

Mapping Cardinality Constraints

- Express the number of entities to which another entity can be associated via a relationship set.
- Most useful in describing binary relationship sets.
- For a binary relationship set the mapping cardinality must be one of the following types:
 - One to one
 - One to many
 - Many to one
 - Many to many

Database Management Systems | Partha Pratim Das

Now, there are certain cardinality constraints based on how many entities in one relate once entity set in the relationship is connected to how many entities in the other entity set that it is connected to and there are different forms let us go one by one.

(Refer Slide Time: 23:46)

Mapping Cardinalities

(a) Many to one

(b) Many to many

Note: Some elements in A and B may not be mapped to any elements in the other set.

Database Management Systems | Partha Pratim Das

So, the first type is one to one which says that every entity in set A is connected to exactly one entity in set B. Now, this could be partial this could be total which means that it could be one to one total which means that every element in A is related to one unique element in B uniqueness is the requirement of one to one. And every element in B is connected to a unique element in A.

Now, it may be possible that some elements in A do not have a relationship with any element in B or some element in B does not have a relationship with an element in A. Then we will say it is partial in terms of one to one but the basic idea is no two entities can be related to more than one entity. You can have one too many where from the first set to the second one entity in the first set may be related to more than one entity in the second set.

If that happens and you say it is you say one to many. Like if we if we separate, I mean if I say that a person and child. Now, a child has a unique father but a father may have multiple children. So, it is one to many from the father to the child it is one to many one father may have multiple children. But every child will have a unique father it is like that. Similarly, you can it can be the other way it can be many to one.

So, it is just the ordering if you if you change the ordering one to many will become many to one. And there could be there could be many to many relationships. There are course and student and the relationship is takes every student takes multiple courses every course is taken by multiple students so it is many to many. So, these are the cardinality mappings that you have.

(Refer Slide Time: 26:06)

Weak Entity Sets (2)

- Since a weak entity set does not have primary key, it cannot independently exist in the ER Model
- It features in the model in relationship with a strong entity set. This is called the **identifying relationship**
- Primary Key of Weak Entity Set
 - The combination of discriminator and primary key of the strong entity set makes it possible to uniquely identify all entities of the weak entity set
 - Thus, this combination serves as a primary key for the weak entity set.
 - Clearly, this primary key is not formed by the weak entity set completely.
 - **Primary Key of Weak Entity Set = Its own discriminator + Primary Key of Strong Entity Set**
- Weak entity set must have **total participation** in the identifying relationship. That is all its entities must feature in the relationship

Database Management Systems | Parth Pratim Das | 37

There is a concept of a weak entity set we know we have learned that the entity set will have a will be distinguishable and will have a primary key more formally these entity sets are called strong entity sets. Now, if I have an entity set where the there is no primary key. But maybe there is a set of attributes typically called discriminator which uniquely identify a group of entities in that set. So, these are typically called weak entity set. So, if I have a weak entity set

naturally, I cannot work with that independently. So, weak entity sets feature in relationship with a strong entity set to see what is.

(Refer Slide Time: 27:09)

Weak Entity Sets (3): Example

- **Strong Entity Set:** *Building*(building_no, building_name, address). *building_no* is its primary key
- **Weak Entity Set:** *Apartment*(door_no, floor). *door_no* is its discriminator as *door_no* alone can not identify an apartment uniquely. There may be several other buildings having the same door number
- **Relationship:** *BA* between *Building* and *Apartment*
- By **total participation** in *BA*, each apartment must be present in at least one building
- In contrast, *Building* has **partial participation** in *BA* only as there might exist some buildings which has no apartment
- **Primary Key:** To uniquely identify any apartment
 - First, *building_no* is required to identify the particular building
 - Second, *door_no* of the apartment is required to uniquely identify the apartment
- Primary key of Apartment = Primary key of Building + Its own discriminator
= *building_no* + *door_no*

Database Management Systems Partha Pratim Das

Let me first go to the example let us say. So, let us say I have an entity set building which has building number, building name, address. Building number is unique so this is a primary key and is a strong entity set. I have another entity set called apartment which has a door number and the floor. Now, this door number is not a primary key of the apartment. Because the same door number may be there in apartments in multiple buildings.

Door number 201 may exist in multiple buildings. So, just by door number you cannot uniquely identify the apartment. Now, let us suppose that there is a relationship say call it BA between the building and apartment which associates that which says that which apartment every apartment which building does it belong to. And naturally every apartment can belong to only one building but a building will have many apartments in it.

So, from building to apartment, it is many to one we can have that relationship. Now, we would also need that in BA the apartment has a total participation. Because it is not separately identifiable an apartment cannot exist without the building being there. So, if an apartment exists it must have a building. So, this is called total participation which means that every entity in the apartment set must have an association through BA relationship to a building entity.

Whereas for a building it is a partial participation because I can have a building which does not yet have apartments. Interiors have not been done the walls have not been made partitioning has not been done. So, there could be buildings without apartments but there cannot be

apartments without building. So, apartment to building is a total participation and that is the requirement of a weak entity set.

Because unless you have total participation since weak entity set just the door number cannot identify every apartment uniquely. You need the Association of the building to get that unique identity. So, what is that unique identity is the primary key of the strong entity set building. And the discriminator your own discriminator the weak entity sets discriminated which is door number here.

These two together will form the primary key of the weak entity set. So, you can see that actually through the relationship it is borrowing the identity from some other entity set. So, if an entity set is strong it has a primary key which is shown by underline. If an entity set is weak, it has a discriminator which is called the door number.

(Refer Slide Time: 30:27)

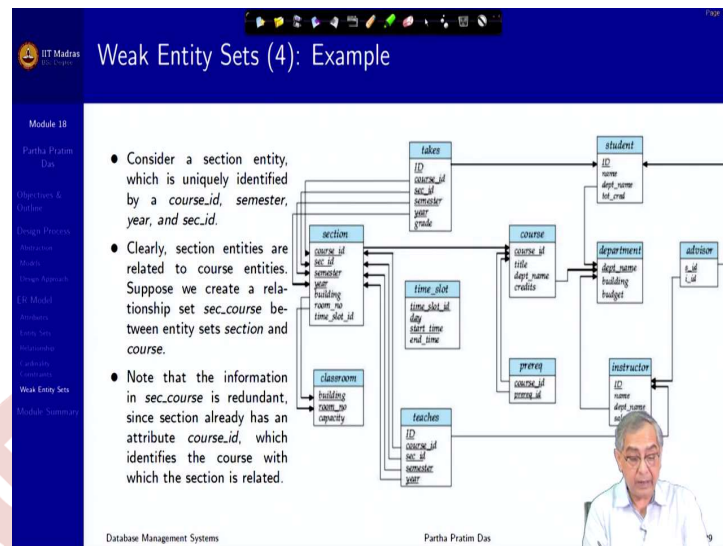
Weak Entity Sets (2)

- Since a weak entity set does not have primary key, it cannot independently exist in the ER Model
- It features in the model in relationship with a strong entity set. This is called the **identifying relationship**
- Primary Key of Weak Entity Set
 - The combination of discriminator and primary key of the strong entity set makes it possible to uniquely identify all entities of the weak entity set
 - Thus, this combination serves as a primary key for the weak entity set.
 - Clearly, this primary key is not formed by the weak entity set completely.
 - **Primary Key of Weak Entity Set = Its own discriminator + Primary Key of Strong Entity Set**
- Weak entity set must have **total participation** in the identifying relationship. That is all its entities must feature in the relationship

Database Management Systems | Parthim Das | 27

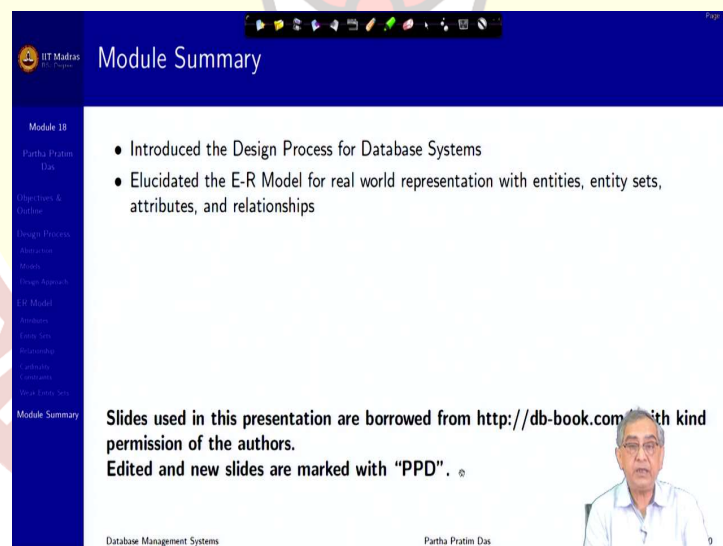
And that relationship I am just going back one slide because I jumped. So, on this relationship BA that you have seen is called the identifying relationship. Because through this relationship every entity in the weak entity set is getting their identity. And the basic two rules are that the weak entity set must have a total participation in the relationship. And the primary key of the weak entity set is a it's own discriminator plus union with the primary key of the strong entity set it is related with.

(Refer Slide Time: 30:58)



We will see that there are weak entity sets have lot of lot of requirements. This is just another example from the course schema that we have been using. I will encourage you to study this and understand.

(Refer Slide Time: 31:10)



So, this comes brings us to the end. So, we have discussed the initial part of the design process of database systems. And getting into the logical modelling and elucidated the in the framework of E-R model and we will continue with this in the next module. Thank you very much for your attention and see you in the next module.